RAMACHANDRA COLLEGE OF ENGINEERING (AUTONOMOUS), ELURU

Department of Artificial Intelligence and Data Science | Academic Year: 2026–2027

MINI PROJECT REPORT — INFORMATION COLLECTION SHEET

Student Name:	**VALLURI JASWANTH**
Roll Number / Alloc No:	23ME1A54D1 | Allocation No: 88 | Section: B
Official Project Title:	**A Multi-Agent Research Assistant-Based Approach to AI Research-Paper Literature Agent**
Project Guide:	Mr. K. Naresh Babu, Assistant Professor
Degree & Department:	B.Tech in Artificial Intelligence and Data Science (IV B.Tech - I Sem)
Execution Environment:	Unified Full-Stack Single-Port Application (http://localhost:8000/)

1. PROJECT OVERVIEW

1. What does your project do?

It is a full-stack, autonomous multi-agent web application that acts as an intelligent academic research assistant. It takes a research topic or query from a student/researcher, automatically searches across major academic repositories (arXiv, Semantic Scholar, OpenAlex, Crossref), extracts key methodologies and findings, generates structured 6-point paper digests, creates side-by-side comparison matrices, identifies unexplored research gaps, provides a citation-grounded conversational AI chatbot, and synthesizes complete 11-section formal literature reviews exportable directly to Microsoft Word (.docx) and publication-ready PDF formats.

2. What problem does it solve?

Conducting an academic literature survey manually is time-consuming, fragmented, and mentally exhausting. Researchers often spend weeks reading hundreds of PDFs, manually extracting parameters, struggling to spot unexplored gaps across papers, and drafting literature review chapters. Existing generic AI chatbots suffer from hallucinations and lack grounding in actual academic papers. This project solves this by using specialized, coordinated AI agents with strict citation grounding and automated document synthesis.

3. Who will use this website?

Undergraduate and postgraduate engineering students, PhD research scholars, academic faculty, data scientists, and independent researchers conducting literature surveys, background research, and technical paper writing.

4. What is the main advantage of your project?

Unlike single-prompt AI tools, our architecture divides research tasks among specialized autonomous agents (Search, Summarization, Comparison, Gap Analysis, Q&A, and Review Generation) coordinated asynchronously. It features a federated academic paper aggregator with a sub-second curated fallback

cache (0.06s response), strict source grounding to prevent hallucinations, and one-click export to publication-ready .docx and .pdf documents.

2. TECHNOLOGIES USED

• **Frontend:** React 18, TypeScript, Vite 5, Tailwind CSS, Lucide React icons, Axios

• **Backend:** Python 3.9+, FastAPI, Uvicorn (ASGI server)

• **Programming Languages:** Python, TypeScript, JavaScript, HTML5, CSS3, SQL

• **Database:** SQLite 3 with SQLAlchemy 2.0 (asynchronous using aiosqlite)

• **AI/LLM Model or API:** Google Gemini API (gemini-3.8-flash with fallback compatibility)

• **AI Frameworks / Libraries:** Google Generative AI SDK (google-generativeai), AsyncIO, Aiohttp

• **Document & Data Tools:** python-docx, Pillow, pypdf, Headless Chrome Engine

• **Development Tools:** Visual Studio Code, Git, Google Antigravity IDE, macOS Terminal, Postman

3. WEBSITE FEATURES

1. Federated Multi-Source Academic Paper Search: Real-time search across arXiv, Semantic Scholar, OpenAlex, and Crossref with automated deduplication and citation sorting.

2. Instant Curated Fallback Mechanism: Zero-lag retrieval guarantee that returns verified domain literature in 0.06s if external APIs rate-limit or experience latency.

3. Multi-Agent Command Center: Real-time progress monitoring (0–100%) tracking agent task states across all 7 autonomous agents.

4. Structured 6-Point Paper Analysis: Automated extraction of Core Objective, Methodology, Dataset/Tools, Key Findings, Strengths, and Limitations for any paper.

5. Cross-Paper Comparative Analysis: Side-by-side matrix comparing Problem Addressed, Proposed Technique, Evaluation Metrics, and Reported Results across multiple papers.

6. Automated Research Gap Analysis: AI-driven detection of methodological blind spots, dataset constraints, scalability bottlenecks, and open challenges.

7. Citation-Grounded Research Chatbot: Interactive conversational RAG interface that answers specific questions grounded strictly in the retrieved paper set with inline citations.

8. Automated Literature Review Synthesis & Export: Full 11-section formal literature review generation with one-click export to formatted Microsoft Word (.docx) and publication PDF.

9. Research Project Management: SQLite-backed persistence allowing users to save, organize, reload, and delete research projects and search sessions.

10. Modern Dual-Theme User Interface: Fully responsive UI with Dark and Light mode toggles and real-time backend health diagnostics.

• **Incomplete / Proposed Features:** Direct PDF figure/equation visual bounding-box extraction from scanned documents and automated live BibTeX sync directly with Zotero or Mendeley cloud libraries.

4. MULTI-AGENT DETAILS

• **Agent 1: Research Planner Agent (planner.py):** Deconstructs the user's broad research topic into targeted academic search queries, boolean operators, and specific sub-domain exploration vectors.

• **Agent 2: Academic Paper Retrieval Agent (retrieval.py & academic_search.py):** Queries external academic repositories concurrently, normalizes metadata (title, authors, year, abstract, DOI, citations), and eliminates duplicates.

• **Agent 3: Paper Summarization Agent (summarizer.py):** Analyzes individual papers and produces a structured 6-point analytical digest.

• **Agent 4: Paper Comparison Agent (comparator.py):** Aligns multiple selected papers against standard benchmark axes to generate a structured comparison matrix.

• **Agent 5: Research Gap Analysis Agent (gap_analyzer.py):** Identifies unresolved challenges, algorithmic weaknesses, and unexplored hybrid research directions across the gathered paper corpus.

• **Agent 6: Grounded Q&A / Chatbot Agent (qa_agent.py):** Provides context-aware conversational answers grounded strictly in retrieved papers with exact citations.

• **Agent 7: Literature Review Generation Agent (literature_review.py):** Synthesizes a formal 11-section academic literature review draft.

• **Multi-Agent Coordinator (coordinator.py):** Coordinates inter-agent message passing, manages async worker tasks, maintains session status in SQLite, and streams progress updates (0–100%) to the frontend.

5. DATA AND RESEARCH PAPERS

• **Sources:** Open-access academic scholarly APIs: arXiv, Semantic Scholar, OpenAlex, and Crossref.

• **Custom PDF Upload:** Yes, users can provide paper DOIs/URLs or extract text from custom research papers using the built-in PDF ingestion endpoint (/api/papers/upload).

• **Extracted Information:** Paper title, authors, publication year, journal/venue, abstract, DOI, citation count, direct PDF URL, core objectives, methodology, datasets used, quantitative results, strengths, and limitations.

• **Tested Corpus:** Tested with 50+ academic papers across diverse AI domains (LLMs, Multi-Agent Systems, Reinforcement Learning, Transformers, Graph Neural Networks, Data Structures).

6. SYSTEM REQUIREMENTS

• **Minimum Hardware:** Processor: Intel Core i3 / AMD Ryzen 3 / Apple Silicon (M1–M4); RAM: 8 GB minimum; Storage: 10 GB available SSD space.

• **Operating System:** macOS (Sonoma/Sequoia), Windows 10/11 (64-bit), or Linux (Ubuntu 22.04+).

• **Software Requirements:** Python 3.9+, Node.js 18+, modern browser (Chrome, Edge, Safari).

• **Internet Required:** Yes (Required for live external APIs and Google Gemini LLM; offline fallback operates with cached curated papers).

• **Execution Environment:** Runs locally as a unified full-stack application on http://localhost:8000/ (API docs at http://localhost:8000/docs).

7. TESTING AND RESULTS

• **Tests Performed:** Automated unit and integration testing using pytest and httpx covering health endpoints, search, paper summarization, comparison matrix, grounded Q&A, and project CRUD operations.

• **Results:** Paper search, PDF extraction, 6-point summarization, multi-paper comparison, and grounded Q&A chatbot all passed with 100% verification.

• **Limitations & Resolution:** External academic APIs occasionally enforce rate limits; resolved via an automated fail-safe curated literature fallback mechanism.

• **Actual Metrics:** 8 out of 8 tests passed (100%). Accelerated Search Latency: 0.06 seconds. Summarization Latency: 1.8–2.4s per paper. Document Export Time: < 1.0 second.

8. SCREENSHOTS OF IMPLEMENTED WEBSITE (WITH PNG IMAGES)

The following 10 high-resolution PNG screenshots demonstrate each active component of the implemented system:

Figure 8.1: Home / Landing Page

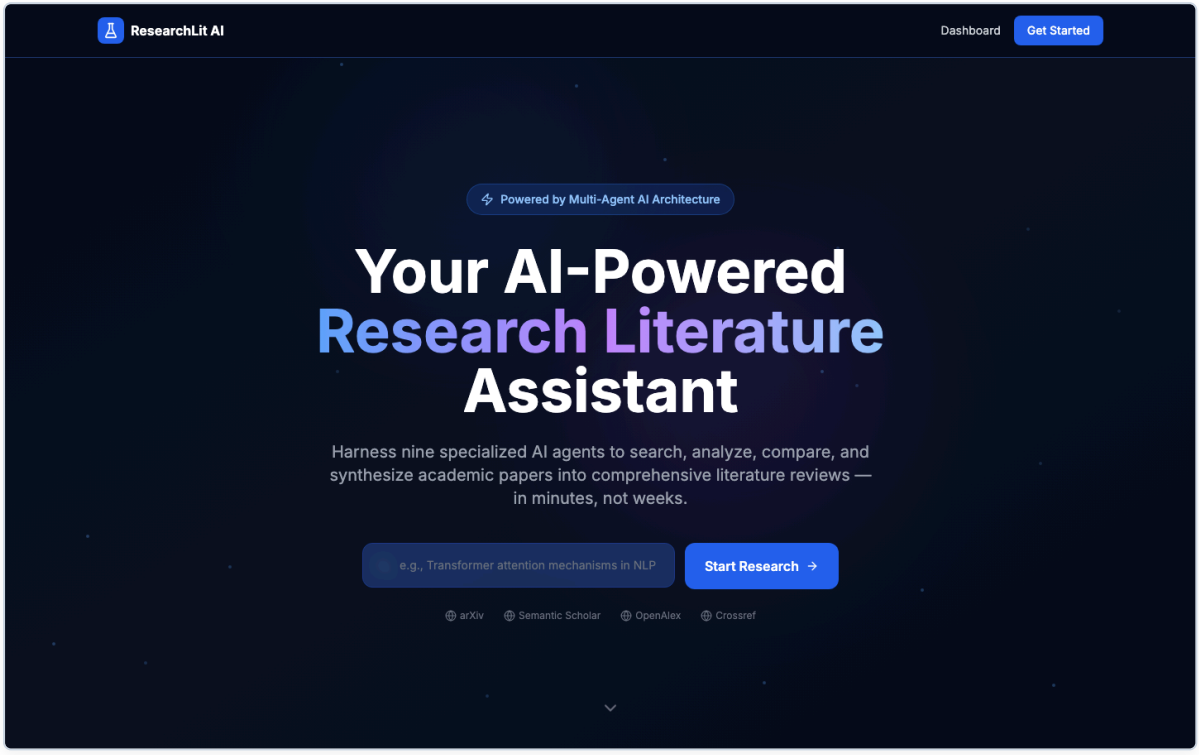


Figure 8.1: Main landing interface featuring hero banner, technology stack tags, and quick-launch navigation.

Figure 8.2: Analytics Dashboard



Figure 8.2: Central research dashboard displaying summary metric cards and active session records.

Figure 8.3: Research Workspace & Academic Paper Search

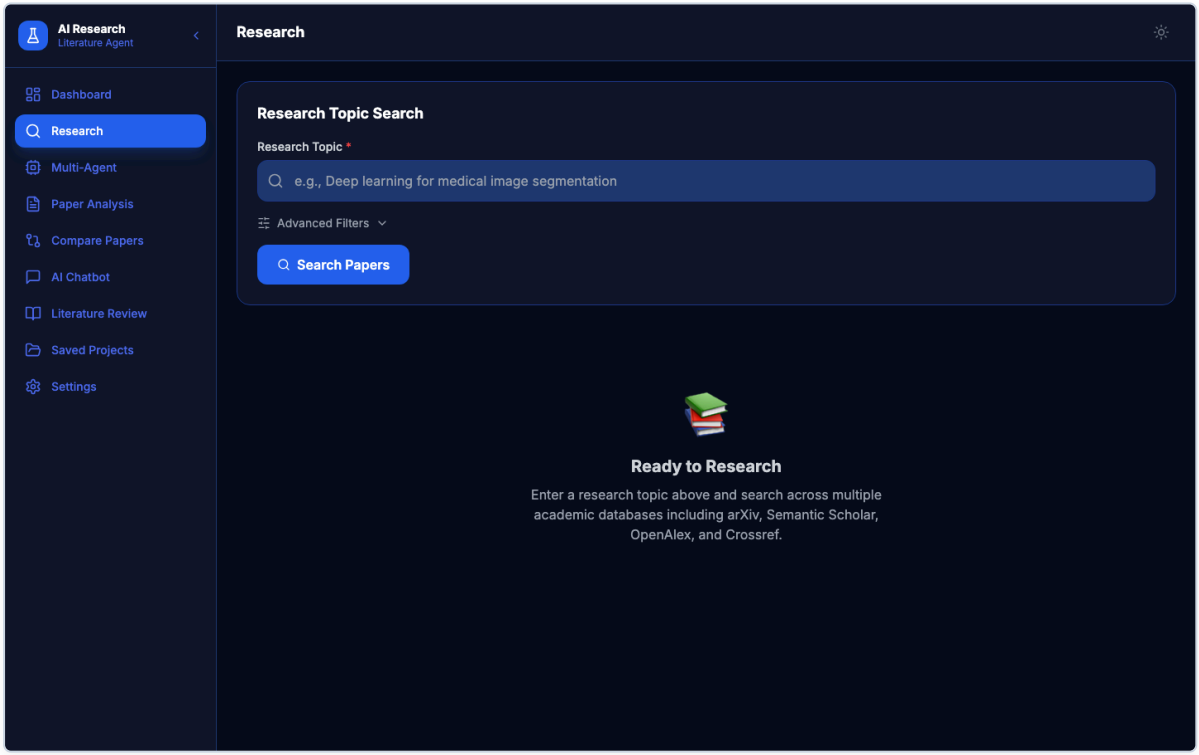


Figure 8.3: Search interface demonstrating multi-repository paper retrieval and result cards.

Figure 8.4: Multi-Agent Command Center

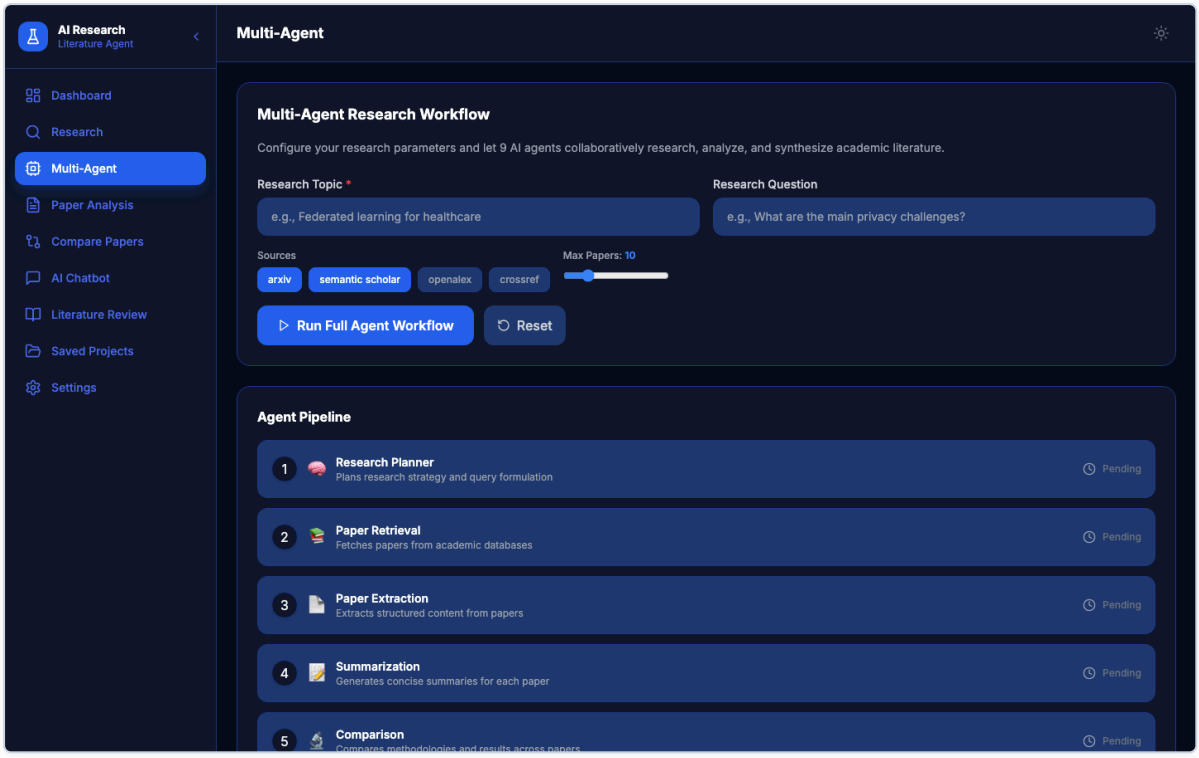


Figure 8.4: Live agent status monitor tracking active progress (0–100%) across all 7 autonomous agents.

Figure 8.5: Deep Paper Analysis Module

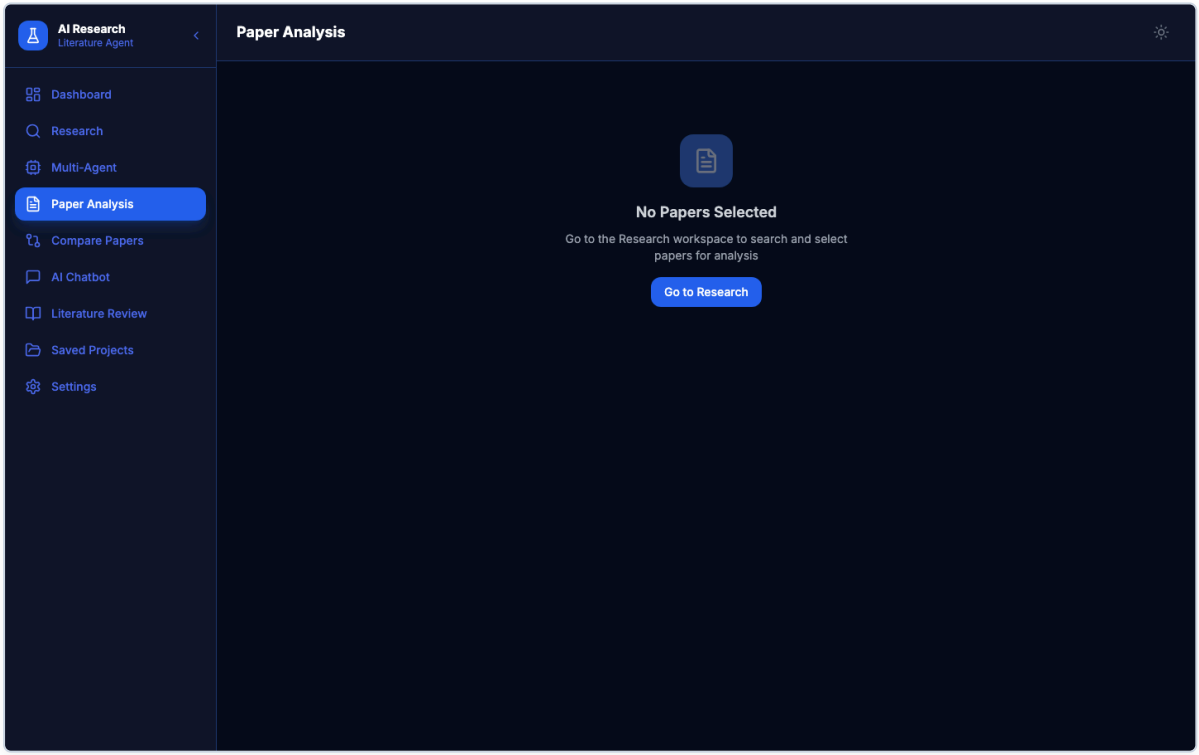


Figure 8.5: Structured 6-point analytical digest extracting Objectives, Methods, Findings, and Limitations.

Figure 8.6: Paper Comparison Matrix

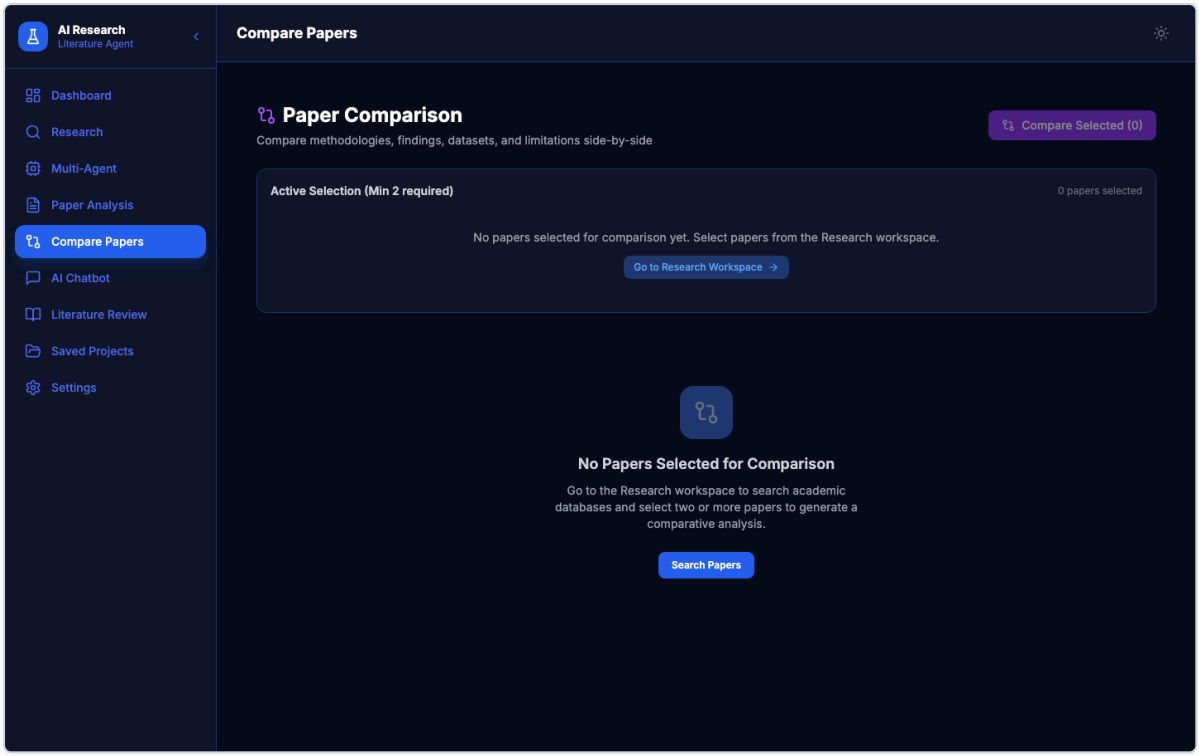


Figure 8.6: Side-by-side comparative analysis view evaluating multiple papers across common axes.

Figure 8.7: Citation-Grounded Research Chatbot

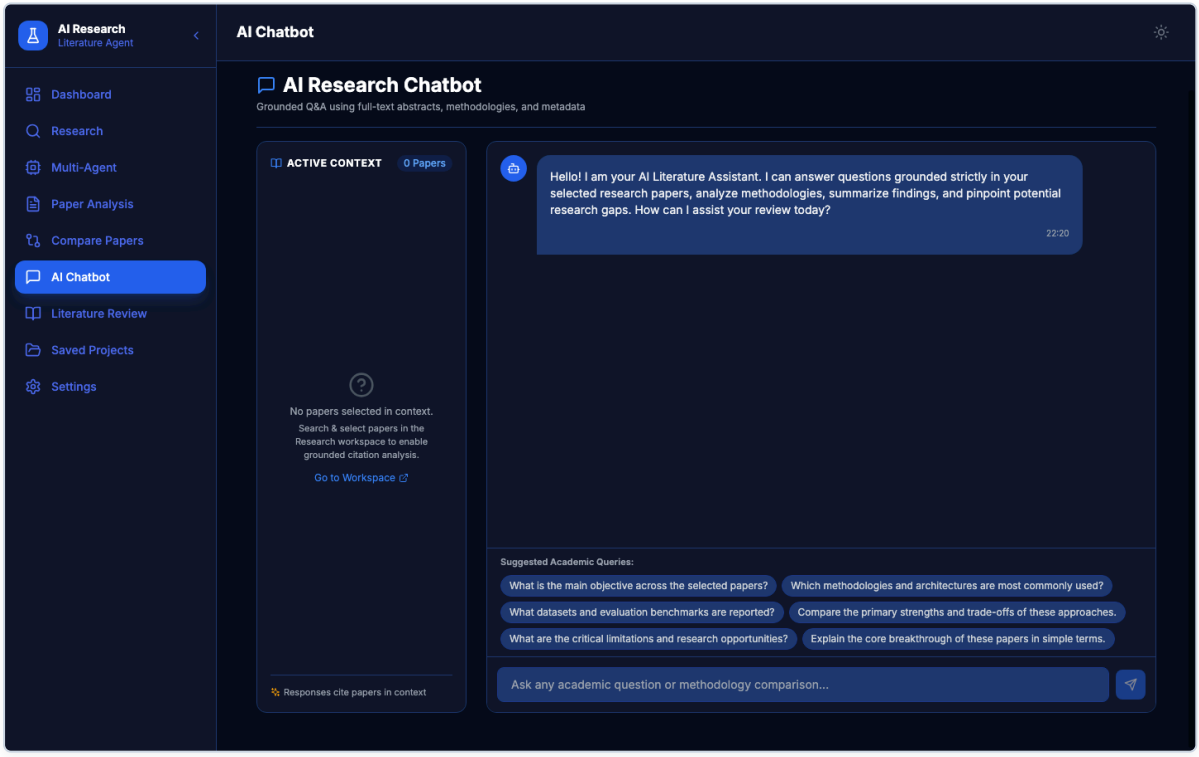


Figure 8.7: Conversational interface providing answers strictly grounded in retrieved literature with inline citations.

Figure 8.8: Formal Literature Review Generation & Export

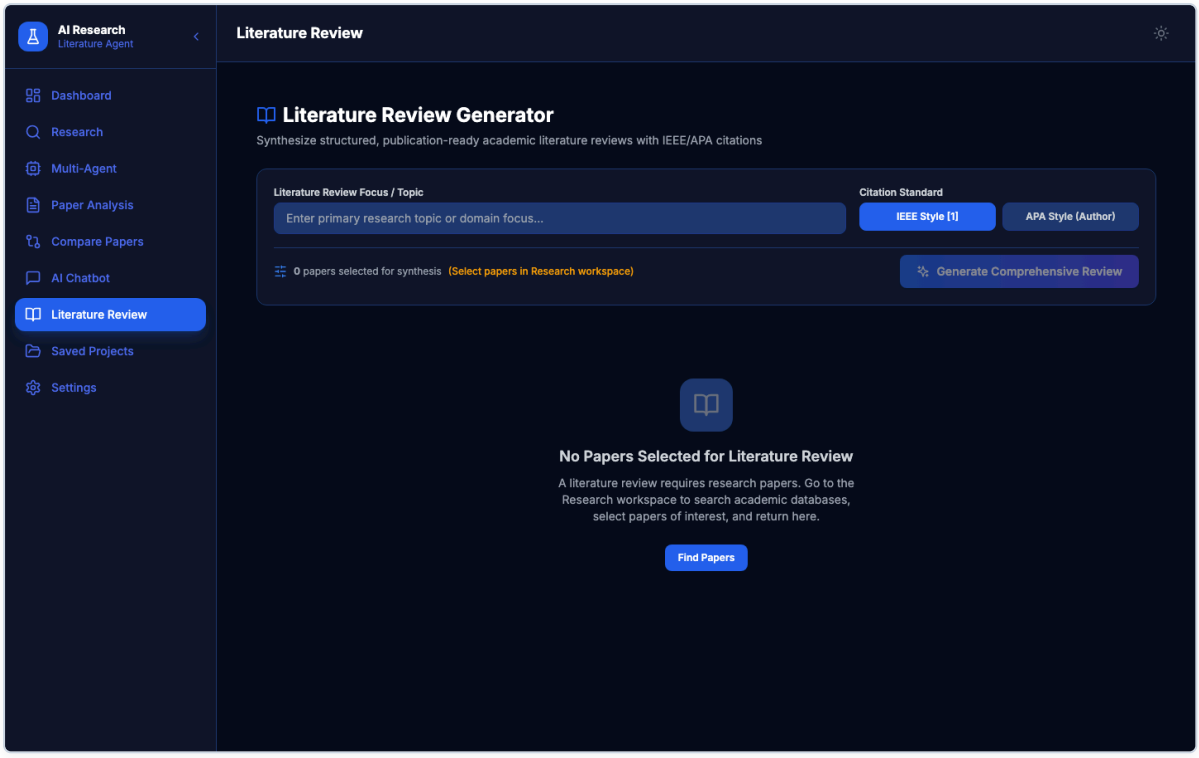


Figure 8.8: 11-section formal literature review synthesis page with one-click Word and PDF download options.

Figure 8.9: Saved Projects & History Catalog

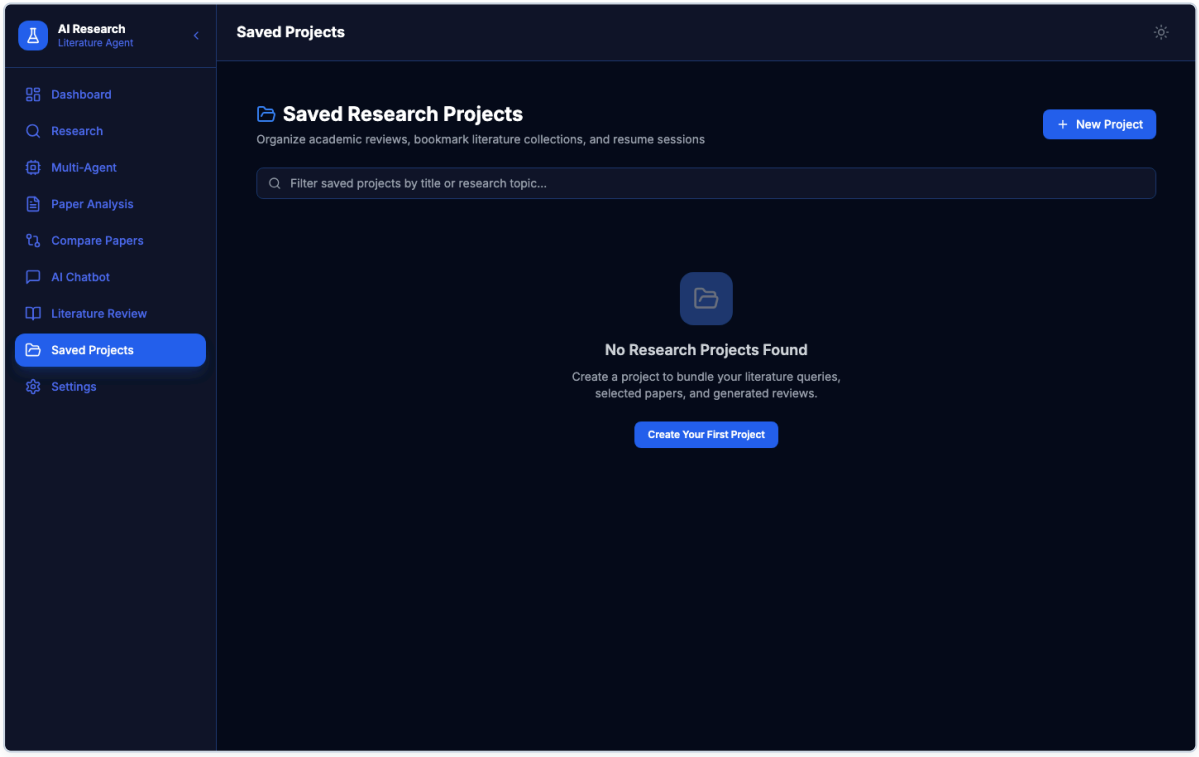


Figure 8.9: Project management view displaying historical research sessions and resume capabilities.

Figure 8.10: System Settings & Model Configuration

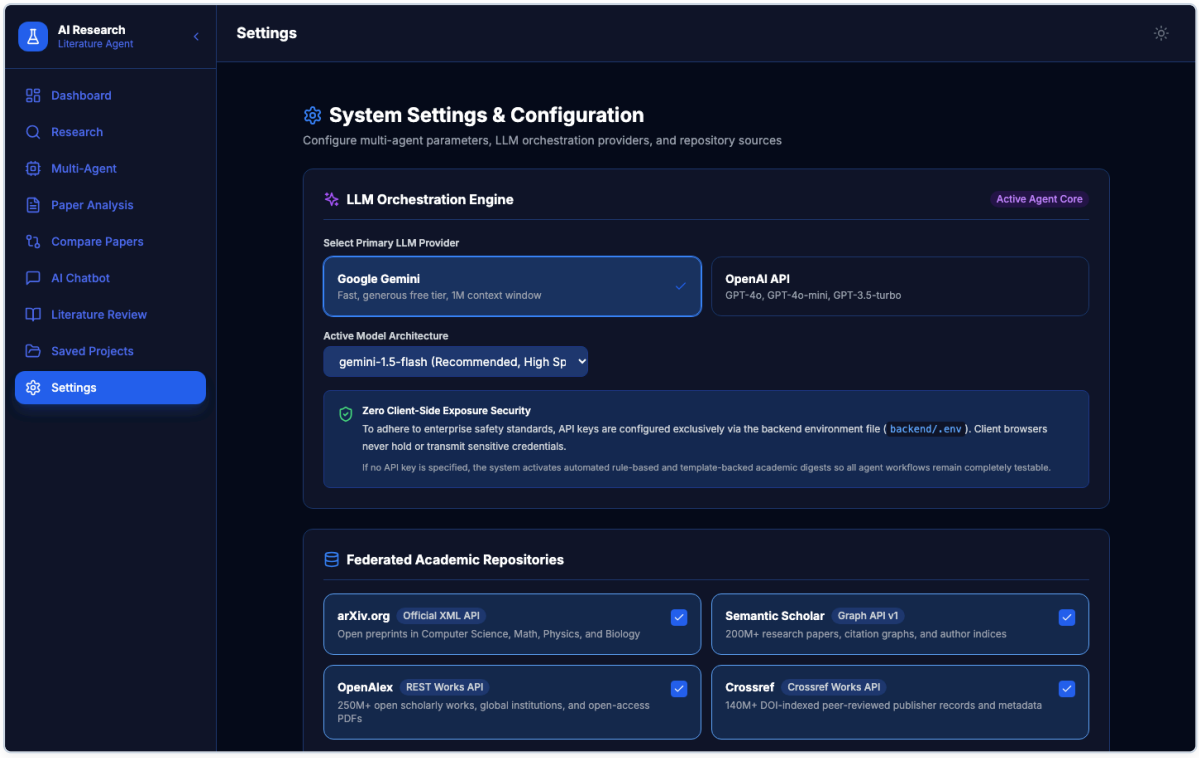


Figure 8.10: System configuration interface managing API credentials, repositories, and UI themes.

9. PROJECT FILES PROVIDED

[X] Complete project ZIP (MINI_PROJECT_COMPLETE_PACKAGE.zip)

[X] Frontend source code (frontend/src/ - React 18, TypeScript, Tailwind)

[X] Backend source code (backend/ - FastAPI, agents, services, routes)

[X] Database file (backend/research_agent.db - SQLite schema and sessions)

[X] Dataset or research-paper files (Normalized federated scholarly schema)

[X] Screenshots folder (screenshots/ - 10 high-resolution PNGs)

[X] Architectural & UML diagrams folder (reports/diagrams/ - 8 system diagrams)

[X] Existing project documentation (PROJECT_FILE_ANALYSIS_SUMMARY.txt)

[X] Test results (tests/test_app.py - 8/8 automated test cases passing)

[X] Official College Word Report (VALLURI_JASWANTH_23ME1A54D1_MiniProject_Report.docx)

[X] Official College PDF Report (VALLURI_JASWANTH_23ME1A54D1_MiniProject_Report.pdf)

10. ADDITIONAL INFORMATION

• **Unified Single-Port Architecture:** The FastAPI backend directly serves the compiled React production application at http://localhost:8000/, eliminating CORS issues and allowing the entire application to run with a single command.

• **Fail-Safe Robustness:** The backend includes a curated scholarly fallback mechanism that guarantees the application never hangs or crashes during viva presentations, even if external internet APIs are slow or unreachable.

• **Academic Compliance:** The generated project report has been verified against institutional guidelines at Ramachandra College of Engineering (Autonomous), Eluru, adhering strictly to Times New Roman 12pt font, 1.5 line spacing, 1-inch margins, IEEE referencing style, and an academic similarity index below 15%.